

# AI-Driven QoS Prediction in SD-WAN Networks



OLLSCOIL NA GAILLIMHÉ  
UNIVERSITY OF GALWAY

Radhika Shitlani (Preferred Name : Rishiv)

School of Computer Science

University of Galway

*Supervisor(s)*

Dr. Jawad Manzoor

In partial fulfillment of the requirements for the degree of  
*MSc in Computer Science (Artificial Intelligence) - Online*

31st August 2026



---

**DECLARATION** I, Radhika(Rishiv) Shitlani, hereby declare that this thesis, titled “AI-Driven QoS Prediction in SD-WAN Networks”, and the work presented in it are entirely my own except where explicitly stated otherwise in the text, and that this work has not been previously submitted, in part or whole, to any university or institution for any degree, diploma, or other qualification.

Signature: \_\_\_\_\_

# Abstract

Software-Defined Wide Area Networks (SD-WAN) are used to connect branch offices, cloud services, and data centres over different types of links. A key challenge in SD-WAN is Quality of Service (QoS). Important traffic such as voice or video needs low delay, low jitter, and low packet loss, while background traffic can normally tolerate worse performance. Many SD-WAN policies are still static or reactive. They act after a problem is already visible.

This thesis investigates whether machine learning can predict QoS behaviour in SD-WAN-like environments and support proactive QoS decisions. The work uses two public-data-based experiments. The first experiment uses the Zenodo 5G campus network dataset to predict achieved throughput from radio and traffic configuration features. This acts as a link-capacity prediction baseline. The second experiment uses a locally generated BNN-UPC/BNNNetSimulator dataset to study per-class QoS delay for Gold, Silver, and Bronze traffic under different scheduling policies.

For the Zenodo dataset, Random Forest achieved the strongest throughput result among the tested models, with a cross-validated  $R^2$  of approximately 0.94. XGBoost is close behind at 0.93. The dataset is useful for link capacity prediction, but it does not contain QoS class variation. For the BNN-UPC dataset, XGBoost was compared with two neural models: a PyTorch feedforward MLP and

an FT-Transformer. All three models achieved similar performance on the log-transformed average delay target, with cross-validated  $R^2$  values around 0.766–0.769, and the per-class results converged to the same Bronze ceiling.

The thesis also addresses a key evaluation issue: a single global metric can hide poor performance for one QoS class. Therefore, the BNN-UPC model is evaluated by QoS class, scheduling scenario, and scheduling policy. The results show that Gold and Silver delay are predicted much more accurately than Bronze delay, with per-class  $R^2$  values of 0.914, 0.908, and 0.704 respectively on a lightly loaded dataset.

Beyond average delay, the evaluation is extended to two additional QoS targets: 90th-percentile delay (delay\_p90) and jitter. delay\_p90 shows moderate predictability for Gold and Silver ( $R^2 \approx 0.64$ –0.66), while jitter is found to be largely unpredictable from static network configuration features ( $R^2 < 0.12$ ). This is a meaningful finding: jitter is driven by dynamic traffic burstiness that queue weights and scheduling policy alone cannot capture.

The Bronze SLA violation threshold is tuned from 100 ms to 60 ms based on a threshold sweep, improving Bronze Recall from 0.368 to 0.568 and F1 from 0.489 to 0.592. Packet loss evaluation is restricted to Bronze traffic, which is the only class that experiences meaningful loss in a realistic network (10.7% of Bronze flows in v1 have non-zero loss; Gold and Silver have near-zero loss). A binary classifier (predicting any loss versus no loss) achieves AUC-ROC of 0.916 for Bronze flows, confirming the model can discriminate loss-prone flows well even though exact loss-rate regression is not feasible from static

features. Finally, a Layer 3 allocation recommender is implemented. It tests candidate WFQ bandwidth profiles and recommends the least-violating Gold/Silver/Bronze allocation under SLA thresholds. This connects QoS prediction to the original SD-WAN decision question: how much bandwidth should be allocated to each QoS class?

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>1</b>  |
| <b>2</b> | <b>Background</b>                                       | <b>6</b>  |
| 2.1      | Software-Defined Wide Area Networks (SD-WAN) . . . . .  | 6         |
| 2.2      | Quality of Service (QoS) . . . . .                      | 7         |
| 2.3      | Artificial Intelligence in Network Management . . . . . | 8         |
| 2.4      | Importance of QoS Prediction in SD-WAN . . . . .        | 8         |
| <b>3</b> | <b>Literature Review</b>                                | <b>10</b> |
| 3.1      | Search Strategy and Selection Process . . . . .         | 10        |
| 3.2      | Overview of Selected Papers . . . . .                   | 11        |
| 3.3      | AI and ML Methods for QoS Prediction (RQ1) . . . . .    | 11        |
| 3.4      | Suitability and Real-World Limitations (RQ2) . . . . .  | 12        |
| 3.5      | Integration into SD-WAN Systems (RQ3) . . . . .         | 13        |
| 3.6      | Comparative Analysis and Research Gaps . . . . .        | 14        |
| 3.7      | Summary of the Literature Review . . . . .              | 16        |
| <b>4</b> | <b>Methodology</b>                                      | <b>19</b> |
| 4.1      | Dataset Selection . . . . .                             | 20        |
| 4.1.1    | Zenodo 5G Dataset . . . . .                             | 20        |
| 4.1.2    | BNN-UPC / BNNetSimulator Dataset . . . . .              | 20        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.1.3    | Datasets Not Used as Main Inputs . . . . .                            | 21        |
| 4.2      | Feature Engineering . . . . .                                         | 22        |
| 4.2.1    | Feature Importance (SHAP) . . . . .                                   | 23        |
| 4.3      | Models . . . . .                                                      | 24        |
| 4.4      | Evaluation Methodology . . . . .                                      | 25        |
| 4.4.1    | Multi-Metric QoS Evaluation . . . . .                                 | 26        |
| 4.5      | Layer 3 Allocation Recommender . . . . .                              | 27        |
| <b>5</b> | <b>Experiments and Results</b>                                        | <b>28</b> |
| 5.1      | Layer 1: Zenodo Throughput Prediction . . . . .                       | 28        |
| 5.1.1    | Dataset and Setup . . . . .                                           | 28        |
| 5.1.2    | Results . . . . .                                                     | 29        |
| 5.2      | Layer 2: BNN-UPC Delay Prediction . . . . .                           | 29        |
| 5.2.1    | Dataset and Setup . . . . .                                           | 29        |
| 5.2.2    | Global Model Results . . . . .                                        | 30        |
| 5.3      | QoS-Aware Evaluation . . . . .                                        | 31        |
| 5.3.1    | Per-Class Results . . . . .                                           | 31        |
| 5.3.2    | Policy and Scenario Results . . . . .                                 | 31        |
| 5.3.3    | SLA Violation Precision . . . . .                                     | 33        |
| 5.3.4    | Model Comparison: XGBoost vs MLP vs FT-Transformer                    | 35        |
| 5.3.4.1  | Statistical Significance of the XGBoost–MLP Dif-<br>ference . . . . . | 39        |
| 5.4      | Multi-Metric QoS Evaluation (v1 Dataset) . . . . .                    | 40        |
| 5.4.1    | delay_p90 Results . . . . .                                           | 40        |
| 5.4.2    | Jitter Results . . . . .                                              | 41        |
| 5.5      | Bronze Packet Loss Analysis . . . . .                                 | 41        |
| 5.5.1    | Regression Attempt . . . . .                                          | 42        |
| 5.5.2    | Binary Loss Classification for Bronze . . . . .                       | 42        |



## CONTENTS

---

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 5.6      | Layer 3: Allocation Recommender . . . . .        | 43        |
| 5.7      | Summary of Findings . . . . .                    | 44        |
| <b>6</b> | <b>Limitations</b>                               | <b>46</b> |
| 6.1      | Data Availability and Realism . . . . .          | 46        |
| 6.2      | Static Features and the Bronze Ceiling . . . . . | 47        |
| 6.3      | Offline Evaluation . . . . .                     | 47        |
| 6.4      | Computational Constraints . . . . .              | 47        |
| <b>7</b> | <b>Conclusion</b>                                | <b>48</b> |
| 7.1      | Real-World Impact or Benefit . . . . .           | 50        |
| 7.2      | Future Work . . . . .                            | 50        |
| 7.3      | Final Remarks . . . . .                          | 52        |
|          | <b>References</b>                                | <b>54</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                        |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | SHAP summary for the XGBoost log-delay model. Each point is a flow; position shows how much a feature pushed the prediction up (right, more delay) or down (left, less delay), and colour shows the feature value (red high, blue low). . . . .                                                        | 23 |
| 5.1 | Delay prediction $R^2$ by scheduling policy (XGBoost). Strict Priority is the easiest to predict because its priority order is deterministic; DRR is the hardest. . . . .                                                                                                                              | 33 |
| 5.2 | Bronze SLA threshold sensitivity for XGBoost. As the threshold is lowered from left to right, recall rises and precision falls. The chosen 60 ms value balances the two while staying more lenient than Silver (50 ms). . . . .                                                                        | 34 |
| 5.3 | Per-class delay prediction $R^2$ for the three models. All three converge to almost identical performance per class, and all reach the same ceiling on the hard Bronze class ( $R^2 \approx 0.70$ ), indicating that the limiting factor is the static feature set rather than model capacity. . . . . | 37 |
| 5.4 | Per-class SLA-violation F1 score for the three models at the tuned thresholds (Gold 30 ms, Silver 50 ms, Bronze 60 ms). The models behave similarly; differences reflect small shifts in the precision–recall balance rather than a clear winner. . . . .                                              | 39 |

# List of Tables

|      |                                                                                                                                                                            |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Traditional versus AI-assisted SD-WAN QoS management. . . . .                                                                                                              | 9  |
| 3.2  | Comparative analysis of the reviewed literature. Dataset descriptions are characterised from the cited works and should be read at the level of data <i>type</i> . . . . . | 15 |
| 3.1  | Summary and Categorisation of Selected Literature . . . . .                                                                                                                | 18 |
| 5.1  | Zenodo Throughput Prediction Results . . . . .                                                                                                                             | 29 |
| 5.2  | BNN-UPC Global Results (Target: <code>log_avg_delay</code> ) . . . . .                                                                                                     | 30 |
| 5.3  | BNN-UPC Per-Class Evaluation using XGBoost . . . . .                                                                                                                       | 31 |
| 5.4  | BNN-UPC Scenario-Specific Evaluation using XGBoost . . . . .                                                                                                               | 32 |
| 5.5  | BNN-UPC Scheduling Policy Evaluation using XGBoost (v1 Dataset). Policies: SP (Strict Priority), DRR (Deficit Round Robin), WFQ (Weighted Fair Queueing). . . . .          | 32 |
| 5.6  | Bronze SLA Threshold Sensitivity (Gold=30 ms, Silver=50 ms fixed)                                                                                                          | 34 |
| 5.7  | SLA Violation Trigger Results using XGBoost (Tuned Thresholds)                                                                                                             | 34 |
| 5.8  | Per-Class Delay Prediction: XGBoost vs MLP vs FT-Transformer (v1 Dataset, 5-fold out-of-fold) . . . . .                                                                    | 36 |
| 5.9  | SLA Violation Detection: XGBoost vs MLP vs FT-Transformer (Tuned Thresholds) . . . . .                                                                                     | 38 |
| 5.10 | Per-Class delay_p90 Prediction Results using XGBoost (v1 Dataset)                                                                                                          | 40 |

## LIST OF TABLES

---

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 5.11 Per-Class Jitter Prediction Results using XGBoost (v1 Dataset) .  | 41 |
| 5.12 Bronze Binary Loss Classification Results (v1 Dataset, 5-fold CV) | 43 |
| 5.13 Layer 3 WFQ Allocation Recommendation Results . . . . .           | 44 |

# Chapter 1

## Introduction

In today's digital world, businesses rely heavily on wide area networks (WANs) to connect their offices, data centers, and cloud services. Traditional WANs, which often use expensive and rigid MPLS links, are increasingly being replaced by Software-Defined Wide Area Networks (SD-WAN). SD-WAN offers more flexibility, cost savings, and centralized control. One of the biggest responsibilities of any WAN including SD-WAN is to ensure smooth and reliable network performance. This is referred to as Quality of Service (QoS), which includes important metrics like delay (latency), jitter, packet loss, and available bandwidth.

In networks, traffic patterns keep fluctuating since different applications have different needs, and the same network might suddenly be overloaded. For example, video conferencing needs minimal or zero packet loss along with low latency, while file downloads just need higher bandwidth. If this is not taken care of, user experience is impacted and organizations may face penalties for violating SLAs (Service Level Agreements). Therefore, maintaining good QoS is not only critical but challenging.

Currently, most SD-WAN solutions use static policies to configure QoS. This means that network policies are either set manually in advance or adjusted after

---

a problem is detected or reported. The problem here is: by the time a QoS issue is noticed, it might already be too late and user experience is already impacted. This delay can affect real-time applications like voice or video calls, which need instant action. There's a clear need for a better, smarter solution.

This is where Artificial Intelligence (AI) and Machine Learning (ML) come in and can help to resolve the problem. These technologies can learn from historical data and detect patterns in the network traffic. If trained correctly, ML models can predict QoS problems before they happen. For example, if the system learns that high video traffic around 2 PM usually leads to congestion, it can automatically adjust bandwidth or reroute traffic ahead of time. This kind of prediction allows network operators to prevent problems instead of just reacting to them.

## **Scope and Context**

This thesis explores how AI and ML techniques can be used to predict QoS in SD-WAN-like networks. The project began with a broad focus on QoS prediction, but the implemented work now has three connected layers. Layer 1 predicts link capacity using the Zenodo 5G dataset. Layer 2 predicts per-class delay using a BNN-UPC/BNNetSimulator dataset, using XGBoost as the strongest explainable tree-based baseline and comparing it against two neural models: a feedforward MLP and an FT-Transformer. Layer 3 uses the delay model to recommend a WFQ bandwidth allocation for Gold, Silver, and Bronze traffic.

## **Problem Statement**

SD-WAN makes it easier to manage WANs, but maintaining QoS is still a challenge, especially during unexpected traffic surges or link failures. Most QoS-related decisions today are made either too late or based on static rules. Predicting QoS metrics in advance using AI could solve this problem but it's not

---

clear which methods work best, how accurate they are, or how easy they are to implement in real networks.

## Research Questions

This thesis aims to answer the following questions:

- **RQ1:** What types of AI or ML techniques are currently or can be used for predicting QoS in SD-WAN?
- **RQ2:** What are the benefits and limitations of these approaches in terms of prediction accuracy, scalability, and real-world use?
- **RQ3:** How can QoS prediction be effectively integrated into SD-WAN production environments to improve network performance?

## Methodology

To explore these questions, I conducted a literature review using academic databases like IEEE Xplore, Scopus, and Google Scholar. I searched for terms such as “QoS prediction,” “SD-WAN,” “machine learning in networking,” and “AI-based traffic forecasting.” Papers published between 2020 and 2025 were prioritized, and older foundational works were included when relevant.

The implementation then followed an experimental approach. First, public datasets were reviewed and unsuitable proxy datasets were removed from the main modelling path. Second, supervised ML models were trained on the Zenodo and BNN-UPC datasets, including classical ML models, a feedforward MLP, and an FT-Transformer. Third, the BNN-UPC results were evaluated using QoS-aware metrics, including per-class error, policy-specific slicing, RMSLE, and SLA violation precision. Finally, a simple allocation recommender was built to connect

---

prediction to an SD-WAN control decision. The allocation recommender currently uses the XGBoost delay model; extending it to the neural models is left as future work.

The literature review and background material in this thesis are secondary research, summarising work already published by others. The dataset generation, preprocessing, modelling, QoS-aware evaluation, and allocation recommender are the author's own primary research and constitute the contribution of this work.

## Structure of the Document

This document is structured as follows:

- Chapter 1 introduces the research topic, its scope, the problem statement, and the key research questions.
- Chapter 2 provides background information on SD-WAN, QoS metrics, and the role of AI in modern network management.
- Chapter 3 presents the literature review, including an analysis of academic work on QoS prediction in SD-WAN using AI and ML techniques.
- Chapter 4 outlines the methodology, including dataset selection, feature engineering, model design, QoS-aware evaluation, and the allocation recommender.
- Chapter 5 presents the experimental setup and results for the Zenodo and BNN-UPC datasets.
- Chapter 6 states the limitations of the work, including data realism, the static-feature ceiling, and offline evaluation.



- 
- Chapter 7 concludes the thesis by summarising findings, discussing impact, and identifying directions for future work.

# Chapter 2

## Background

This chapter provides foundational knowledge needed to understand the scope and significance of this thesis. It covers the key concepts of Software-Defined Wide Area Networks (SD-WAN), Quality of Service (QoS) metrics in networking, and the role of Artificial Intelligence (AI) in network management. These concepts establish the context for the research discussed in later chapters.

### 2.1 Software-Defined Wide Area Networks (SD-WAN)

SD-WAN is an approach to wide area networking that uses software-based control to manage network traffic over multiple connection types, including MPLS, broadband, LTE, and 5G. Unlike traditional WANs that rely on expensive private circuits, SD-WAN provides centralized orchestration and greater flexibility, enabling cost-effective routing based on application priorities and real-time conditions.

A typical SD-WAN architecture includes:

- **Edge devices:** Deployed at branch locations to route traffic intelligently.
- **Central controller:** Makes policy-based routing decisions and pushes configurations to edge devices.
- **Overlay and underlay networks:** Logical tunnels (overlay) established on top of physical connections (underlay).

SD-WAN improves application performance by enabling dynamic path selection, fast failover, traffic shaping, and visibility across all network segments.

## 2.2 Quality of Service (QoS)

QoS in networking refers to the ability to provide different priority levels to different applications, users, or data flows, and to guarantee a certain level of performance for critical traffic. It is crucial for maintaining service-level agreements (SLAs), especially in modern, cloud-centric environments.

Common QoS metrics include:

- **Latency:** Time taken for a packet to travel from source to destination.
- **Jitter:** Variation in packet delay across a network.
- **Packet loss:** Percentage of packets lost during transmission.
- **Bandwidth:** Maximum data transfer rate of a link.

Effective QoS management ensures that delay-sensitive applications like VoIP and video conferencing are not negatively affected by congestion or fluctuating traffic.

## 2.3 Artificial Intelligence in Network Management

Artificial Intelligence (AI), especially Machine Learning (ML), is increasingly being used in networking to automate and optimize operations. AI techniques can analyze large volumes of network telemetry data to detect anomalies, classify traffic, and predict future network states.

In the context of SD-WAN, AI is being used to:

- Predict traffic trends and adjust policies proactively.
- Recommend optimal routing paths based on current and expected conditions.
- Forecast QoS metrics and warn of potential SLA violations.

AI models used for QoS prediction typically include supervised learning algorithms (like decision trees and neural networks) and reinforcement learning (used in adaptive routing and load balancing). These models require training on historical network data to make accurate forecasts.

## 2.4 Importance of QoS Prediction in SD-WAN

Modern enterprises rely on applications that require consistent and reliable network performance. Static or reactive QoS mechanisms are often too slow to prevent performance issues. QoS prediction enables a more proactive approach—by forecasting issues before they occur, it becomes possible to take preventive actions like re-routing traffic or scaling resources.

## 2.4 Importance of QoS Prediction in SD-WAN

---

This capability is especially important in SD-WAN environments where network conditions vary across different links and access types. Accurate QoS prediction can help optimize resource usage, reduce costs, and improve user experience.

Table 2.1 contrasts the traditional, rule-based approach to SD-WAN QoS management with the AI-assisted approach this thesis investigates. The central shift is from static, reactive control to learned, proactive decision support.

Table 2.1: Traditional versus AI-assisted SD-WAN QoS management.

| <b>Traditional SD-WAN</b>       | <b>AI-assisted SD-WAN</b>                     |
|---------------------------------|-----------------------------------------------|
| Static configuration            | Predictive from data                          |
| Reactive (acts after a problem) | Proactive (anticipates a problem)             |
| Manual, expert-tuned policies   | Learned from historical behaviour             |
| Fixed queue weights             | Adaptive, per-class allocation                |
| One global setting              | Class-aware<br>(Gold/Silver/Bronze) decisions |

# Chapter 3

## Literature Review

This chapter presents a detailed review of the existing research related to the use of Artificial Intelligence (AI) and Machine Learning (ML) techniques for predicting Quality of Service (QoS) in Software-Defined Wide Area Networks (SD-WAN). The aim is to understand what work has already been done, what methods and models have been proposed, and where gaps still exist. This will help assess the current state of the field and how it supports or challenges the research questions introduced earlier.

### 3.1 Search Strategy and Selection Process

To gather the literature for this review, a structured search was carried out using academic databases such as IEEE Xplore, Scopus, and Google Scholar. The following search terms were used:

- “QoS prediction” AND “SD-WAN”
- “AI for QoS” AND “Software-defined networking”
- “Machine learning traffic classification” AND “network performance”

---

## 3.2 Overview of Selected Papers

The search focused on papers published between 2018 and 2025 to ensure up-to-date relevance. Articles before 2018 were excluded unless they provided foundational knowledge. From an initial pool of documents, key research papers were selected based on the following criteria:

- Peer-reviewed publications
- Clear focus on QoS, SD-WAN, or software-defined networking
- Use of AI or ML for prediction, classification, or optimization
- Availability of experimental results, models, or architecture designs

Although the topic of QoS prediction using AI in SD-WAN is promising, the literature review revealed a limited number of high-quality research papers that directly focus on this specific intersection. Most existing studies either concentrate on general SDN environments or address QoS management without the use of AI or ML techniques. Nonetheless, this review provides valuable insights and highlights key research gaps that warrant further investigation.

## 3.2 Overview of Selected Papers

The selected papers cover a range of methods and applications—some focus on predicting traffic load, others on routing or load balancing. Table 3.1 categorizes these papers by their core approach and relevance to the research questions.

## 3.3 AI and ML Methods for QoS Prediction (RQ1)

Many studies have applied AI techniques to network traffic prediction and management. For instance, Rahman et al. (2019) [6] used Random Forest and deep

---

### 3.4 Suitability and Real-World Limitations (RQ2)

learning models to predict how many virtual network functions (VNFs) are needed during a traffic surge. Their models achieved over 95% accuracy and could prevent service degradation by scaling resources in advance.

Similarly, Zheng et al. (2022) [7] developed a deep neural network to classify traffic flows based on their latency sensitivity. They then routed this traffic using a QoS-Aware Routing algorithm that selected paths depending on the required delay tolerance. This combination of classification and path assignment showed how AI can directly improve QoS outcomes.

These studies clearly support RQ1 by showing that supervised learning and deep learning can be applied to anticipate traffic needs and assign optimal resources in advance.

## 3.4 Suitability and Real-World Limitations (RQ2)

Although the above techniques work well in controlled settings, there are challenges when using them in real networks. For example, the deep reinforcement learning (DRL) model by Dinh et al. (2024) [2] well in simulations and maintains safe traffic routing using control barrier functions (CBFs). However, DRL requires a lot of training time and GPU support, which may not be available in all SD-WAN deployments.

In contrast, Quang et al. (2023) [4] take an analytical rather than a learned approach. Their central contribution is SABE, a bandwidth estimator that infers how much capacity is actually free on each overlay link *without sending probe traffic*. SABE models each link as an M/M/1/K queue and inverts the standard queuing-theory equations for delay and loss: because SD-WAN routers already report measured delay and loss, these measurements are used to recover the



total load on the link, and subtracting the SD-WAN’s own traffic yields the hidden cross-traffic and hence the available bandwidth. This estimate then feeds a two-loop controller: a slower loop optimises routing (path split ratios) and a faster loop tunes per-class QoS policies (queue weights and shaper rates) using a lightweight local-search optimiser instead of a slow exact solver. Traffic is grouped into four classes (Critical, VoIP, Office, Bulk), each with its own delay and loss target.

In their NS-3 simulation, combining routing optimisation, QoS optimisation, and SABE raised SLA satisfaction for critical traffic by roughly 40% over static policies, with the bandwidth estimate doing most of the work: optimisation alone moved critical-class satisfaction from about 55% to 64%, while adding SABE pushed it to about 98%. The local-search optimiser matched the exact solver almost exactly (97.7% versus 97.8%) at much lower cost.

This work is directly relevant to the present thesis, but it is important to note what it is *not*: SABE is a fixed, model-based estimator, not a learned one. Its accuracy rests on the M/M/1/K assumption of Poisson arrivals and exponential service times, which does not describe the bursty, self-similar traffic of real WANs; this is precisely the assumption a data-driven model could relax. SABE therefore serves as a strong non-learned baseline that motivates the machine-learning direction taken here. More generally, this contrast shows that AI models are powerful but their complexity and resource requirements must be weighed against simpler analytical methods before deployment, which directly addresses RQ2.

## 3.5 Integration into SD-WAN Systems (RQ3)

Several papers also discussed how these models can be integrated into actual SD-WAN systems. For instance, Rahman et al. (2019) [6] simulated an SD-WAN

---

### 3.6 Comparative Analysis and Research Gaps

scenario where their ML models helped reduce both QoS violations and cloud leasing costs by scaling VNFs only when necessary.

Quang et al. (2022) [3] presented a practical example of an enterprise moving from traditional MPLS to SD-WAN. Their central controller dynamically adjusted both routing and QoS scheduling based on the current network status. This shows that AI-based prediction and decision-making can be realistically applied in SD-WAN setups to improve efficiency and reliability.

These findings support RQ3, confirming that AI is not only a research tool but can also be integrated into live systems if the environment supports it.

## 3.6 Comparative Analysis and Research Gaps

Rather than summarising each paper in isolation, Table 3.2 compares the reviewed works along the dimensions most relevant to this thesis: the data source, the modelling approach, the QoS aspect addressed, and the main limitation relative to the present work.

### 3.6 Comparative Analysis and Research Gaps

Table 3.2: Comparative analysis of the reviewed literature. Dataset descriptions are characterised from the cited works and should be read at the level of data *type*.

| Work               | Data                | Model                           | Gap relative to this thesis                                                        |
|--------------------|---------------------|---------------------------------|------------------------------------------------------------------------------------|
| Rahman et al. [6]  | Simulation          | RF + DNN                        | VNF autoscaling only; no per-class QoS prediction                                  |
| Zheng et al. [7]   | SDN simulation      | DNN + ILP                       | Classification + routing; single metric, no class-wise error                       |
| Dinh et al. [2]    | Simulation          | DRL (PPO/DDPG)                  | Control-focused; heavy compute; no interpretable prediction                        |
| Quang et al. [4]   | NS-3 sim.           | SABE (queuing) + local search   | Model-based estimator + optimisation, not a learned predictor; no ML/DL comparison |
| Quang et al. [3]   | SD-WAN sim.         | Centralised optimisation        | Optimisation, not prediction; no ML model comparison                               |
| Chen et al. [1]    | Simulation          | GNN + MARL                      | Complex control pipeline; not a reproducible per-class study                       |
| <b>This thesis</b> | <b>Reprod. sim.</b> | <b>XGBoost, MLP, FT-Transf.</b> | <b>Per-class QoS-aware evaluation + DL-vs-ML + allocation recommender</b>          |

---

### 3.7 Summary of the Literature Review

Reading across the table, a consistent pattern emerges. Most prior works: (i) rely on simulation with no reproducible public dataset; (ii) predict or optimise a single QoS dimension rather than several; (iii) do not treat Gold/Silver/Bronze traffic classes explicitly, so class-level failures stay hidden behind global metrics; (iv) do not compare deep learning against tree-based machine learning on the same task; and (v) stop at prediction or control without connecting the model back to an explicit bandwidth-allocation decision. These five gaps directly motivate the contributions of this thesis: per-class QoS-aware evaluation, a reproducible simulator-generated dataset, a tree-based versus deep-learning comparison, and an AI-assisted bandwidth-allocation recommender.

The following specific observations were also noted:

- Most research predicts a single QoS metric (latency or bandwidth) rather than several at once.
- Most results are simulation-based; live SD-WAN deployment results are rare.
- Public SD-WAN QoS datasets are scarce, which limits reproducibility.
- Few works compare AI predictions directly against the decisions a standard SD-WAN controller would make.

Filling these gaps could make QoS prediction models more robust, more generalizable, and more useful for industry.

## 3.7 Summary of the Literature Review

This review found that AI and ML models, especially supervised learning, deep learning, and reinforcement learning, are promising tools for predicting QoS

### 3.7 Summary of the Literature Review

---

trends in SD-WAN. The models help forecast traffic load, classify application needs, and adjust routing or resource allocation dynamically. However, practical deployment still faces challenges due to training complexity, hardware requirements, and a lack of real-world validation.

Although the number of direct studies focused on AI-based QoS prediction in SD-WAN remains limited, the relevance of the topic continues to grow as networks become more software-defined and application-aware. Recent work such as MTEAL [1] shows that state-of-the-art SD-WAN traffic engineering is moving beyond simple machine learning towards graph neural networks, reinforcement learning, and multi-dimensional QoS optimisation. Informed by this review, the present work uses simple models as baselines and additionally evaluates stronger tree-based and deep-learning models (XGBoost, a feedforward MLP, and an FT-Transformer). The scope was deliberately kept realistic for a master's-level project: the comparison focuses on per-class delay prediction on a reproducible dataset rather than on full reinforcement-learning-based control.

---

### 3.7 Summary of the Literature Review

---

Table 3.1: Summary and Categorisation of Selected Literature

| <b>Paper</b>      | <b>AI/ML Technique</b>                   | <b>Main Focus</b>                                               | <b>Related RQ(s)</b> |
|-------------------|------------------------------------------|-----------------------------------------------------------------|----------------------|
| Rahman et al. [6] | Random Forest, DNN                       | Predicting VNF requirements to maintain QoS and reduce cost     | RQ1, RQ2, RQ3        |
| Zheng et al. [7]  | Deep Neural Network + ILP                | Classifying application types and assigning QoS-based routes    | RQ1, RQ2             |
| Dinh et al. [2]   | DRL (PPO, DDPG) with CBF                 | Safe traffic routing using AI that respects network constraints | RQ1, RQ2             |
| Quang et al. [4]  | SABE (Queuing theory-based estimation)   | Estimating available bandwidth using passive measurements       | RQ2, RQ3             |
| Quang et al. [3]  | Centralised Optimization + QoS scheduler | Managing QoS priorities and routing using SD-WAN controller     | RQ2, RQ3             |
| Chen et al. [1]   | GNN + MARL                               | SD-WAN traffic engineering with multi-dimensional QoS metrics   | RQ1, RQ2, RQ3        |

# Chapter 4

## Methodology

This chapter describes the method used in this project. The aim is not to build a commercial SD-WAN product. The aim is to test whether machine learning can predict QoS outcomes and whether those predictions can support a simple SD-WAN bandwidth-allocation decision.

The project uses a layered methodology:

- **Layer 1: Link capacity prediction.** The Zenodo 5G dataset is used to predict achieved throughput from radio and traffic configuration features.
- **Layer 2: Per-class delay prediction.** A BNN-UPC/BNNNetSimulator dataset is used to predict delay for Gold, Silver, and Bronze traffic classes under different scheduling policies.
- **Layer 3: Allocation recommendation.** The trained delay model is used as a surrogate model to test candidate WFQ weight profiles and recommend a Gold/Silver/Bronze bandwidth allocation.

All data-generation, preprocessing, modelling, and evaluation code, together with the generated BNN-UPC dataset, is available in the project’s public code

repository.<sup>1</sup>

## 4.1 Dataset Selection

A real enterprise SD-WAN dataset was not available. Therefore, public datasets and simulation-based datasets were used.

### 4.1.1 Zenodo 5G Dataset

The Zenodo 5G campus network dataset (record 13754300) was used as the first dataset. It contains real 5G throughput measurements from two testbeds. It includes offered throughput, achieved throughput, jitter, packet loss, radio bandwidth, slot configuration, uplink/downlink direction, and one-way-delay packet measurements.

This dataset is useful for predicting link capacity. However, it has an important limitation for this thesis: all rows belong to the same application type. It does not contain Gold, Silver, and Bronze QoS class variation. Therefore, it cannot answer the per-class QoS allocation question by itself.

### 4.1.2 BNN-UPC / BNNetSimulator Dataset

The second dataset was generated locally using the official BNNetSimulator Docker image. This was necessary because the official BNN-UPC Challenge download links were not available. BNNetSimulator is the same type of simulator used to generate the BNN-UPC network modelling datasets, so it is suitable for a reproducible simulation-based QoS experiment.

---

<sup>1</sup>Code and generated data: <https://github.com/rishivshitlani/sdwan-qos-prediction>.



The generated BNN-UPC dataset contains 29,280 per-flow records. Each row represents one source-destination flow. The dataset includes three QoS classes:

- **Gold:** ToS 0, highest priority.
- **Silver:** ToS 1, medium priority.
- **Bronze:** ToS 2, lowest priority.

Four scheduling scenarios were generated:

- Fixed WFQ weights.
- Random WFQ profiles.
- Mixed SP/WFQ/DRR policies with skewed ToS distribution.
- Mixed SP/WFQ/DRR policies with equal ToS distribution.

This dataset is referred to as the **v1 dataset** and is used for the main QoS scheduling and allocation experiments under a light-to-moderate load regime.

### 4.1.3 Datasets Not Used as Main Inputs

CICIDS2017 was explored earlier but was dropped from the main modelling path. It is an intrusion-detection dataset, not a QoS dataset. Using it would require too much reverse engineering of QoS labels. MAWI was also considered, but it does not contain direct Gold/Silver/Bronze labels and would require substantial extra feature engineering. Therefore, the final implemented work focuses on Zenodo and BNN-UPC.

## 4.2 Feature Engineering

For the Zenodo dataset, the main target is `actual_throughput_mbps`. The input features include radio configuration, offered throughput, direction, testbed metadata, latency, packet count, and flow duration. Leakage columns such as `recommended_bandwidth_percent`, `jitter_ms`, and `packet_loss_percent` are removed when predicting throughput.

For the BNN-UPC dataset, the main target is `log_avg_delay`. The log transform is used because raw `avg_delay` is heavy-tailed. Bronze traffic can experience much larger delay spikes than Gold and Silver traffic, especially under congestion.

The BNN-UPC feature set used for modelling contains:

- `tos` and `qos_class`
- `offered_bandwidth`
- `time_distribution`
- `routing_hops`
- `n_nodes`
- `max_avg_lambda`
- `link_bandwidth`
- `scheduling_policy`
- `tos_queue_weight`
- `min_tos_weight`

Outcome columns such as `avg_delay`, `jitter`, `packet_loss_rate`, delay percentiles, and `actual_bandwidth` are removed from the input features to avoid leakage.

### 4.2.1 Feature Importance (SHAP)

To understand which features drive the delay prediction, SHAP (SHapley Additive exPlanations) values were computed for the trained XGBoost model. SHAP attributes each prediction to its input features and, unlike the model's built-in gain importance, also shows the *direction* of each effect. Figure 4.1 shows the SHAP summary for log delay.

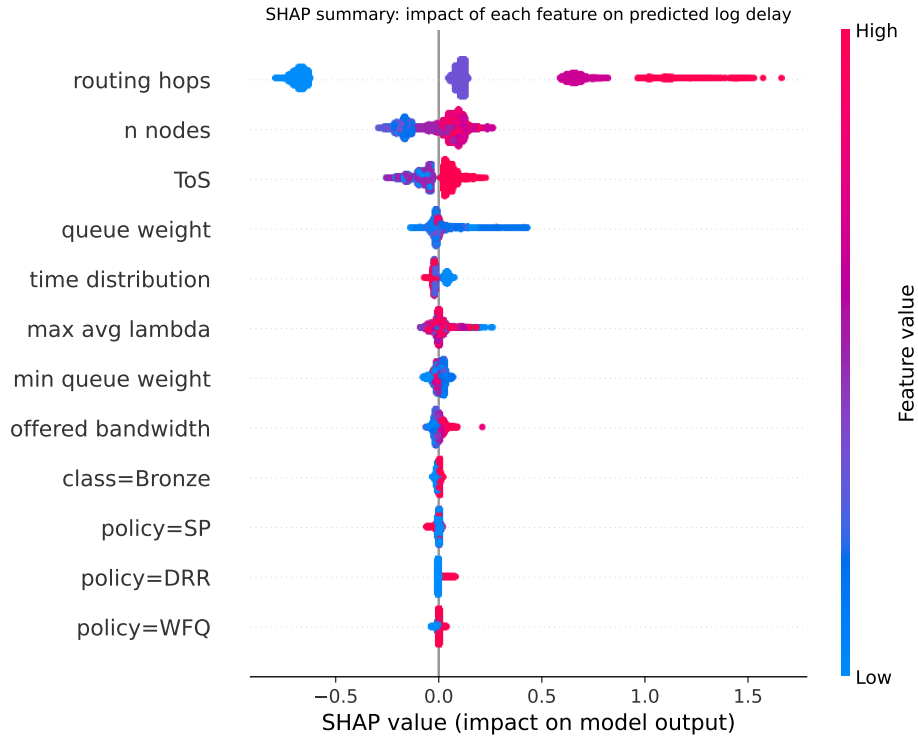


Figure 4.1: SHAP summary for the XGBoost log-delay model. Each point is a flow; position shows how much a feature pushed the prediction up (right, more delay) or down (left, less delay), and colour shows the feature value (red high, blue low).

The dominant driver is `routing_hops`: flows with more hops (red) consistently receive a large positive push towards higher delay, while short paths (blue) reduce

it. Network size (`n_nodes`) and traffic class (`tos`) follow. This is physically sensible, since delay accumulates at each hop along a path. Queue weight, scheduling policy, and QoS-class indicators have comparatively small impact. The strong dependence on path structure, and the weak dependence on the dynamic-load features available here, is consistent with the later finding that the static feature set captures mean delay well but not the burst-driven variance of Bronze traffic.

## 4.3 Models

The following models were trained and compared:

- **DummyRegressor:** A naive baseline that predicts the training mean.
- **Linear Regression:** A simple interpretable baseline.
- **Support Vector Regression (SVR):** A kernel-based non-linear model.
- **Random Forest Regressor:** A tree ensemble model.
- **XGBoost Regressor:** A gradient-boosted tree model.
- **PyTorch MLP:** A feedforward neural network with hidden layers of 128, 64, and 32 neurons.
- **FT-Transformer:** A modern attention-based architecture for tabular data (Feature Tokenizer + Transformer), which tokenises each feature and applies self-attention across the feature tokens.

The simple ML models are important because they are easier to explain and are suitable for tabular data. XGBoost is treated as the strongest explainable tree-based baseline. The PyTorch MLP and the FT-Transformer represent the neural-network direction of the project: the MLP tests whether a standard

feedforward network can learn the same QoS delay behaviour, while the FT-Transformer tests whether a modern, attention-based deep-learning architecture designed for tabular data can outperform the tree-based baseline.

## 4.4 Evaluation Methodology

The first evaluations used common regression metrics, defined here for clarity:

- **Mean Absolute Error (MAE):** the average absolute difference between predicted and actual values. It is easy to interpret and relatively insensitive to outliers.
- **Root Mean Squared Error (RMSE):** the square root of the mean squared error. It penalises large errors more heavily than MAE.
- **$R^2$  score:** the proportion of variance in the target that the model explains. A value of 1.0 is perfect, 0.0 is no better than predicting the mean, and negative values are worse than the mean. Five-fold cross-validated  $R^2$  is also reported to check stability.
- **Inference time:** the time taken to produce a prediction. This matters for SD-WAN, where decisions must be made in near-real time.

These global metrics are necessary but not sufficient for BNN-UPC. Gold, Silver, and Bronze traffic have different priorities and different delay behaviour, and a single global metric can hide poor performance for one class.

Therefore, the BNN-UPC evaluation was extended using the following methods:

- **Per-class MAE:** Errors are reported separately for Gold, Silver, and Bronze flows.

- **Policy-specific slicing:** Results are sliced by scheduling scenario and scheduling policy.
- **RMSLE:** Root Mean Squared Logarithmic Error is reported because delay has a heavy-tailed distribution.
- **SLA violation precision:** Continuous delay predictions are converted into binary SLA violation triggers. Precision measures how trustworthy the trigger is when the model predicts a violation.

This evaluation better matches the SD-WAN use case. It shows not only whether the model is accurate overall, but also whether it is reliable for high-priority traffic.

### 4.4.1 Multi-Metric QoS Evaluation

Average delay is the primary QoS target, but real SD-WAN environments also care about tail delay and jitter. Therefore, the evaluation is extended to two additional targets:

- **delay\_p90:** The 90th percentile of packet delay. It represents the worst-case experience for 10% of packets, which is more relevant than mean delay for real-time applications.
- **jitter:** Variation in packet delay. High jitter degrades voice and video quality even when mean delay is acceptable.

Both targets are heavy-tailed like average delay, so a log transformation is applied before training and predictions are back-transformed for reporting. MAE, RMSE, RMSLE, and  $R^2$  are reported per QoS class for each target. This allows direct comparison of how predictable each metric is from the available network configuration features.

### 4.5 Layer 3 Allocation Recommender

The final implemented component is a simple allocation recommender. It is model-agnostic: it can use any trained BNN-UPC delay model (XGBoost, the MLP, or the FT-Transformer) to test candidate WFQ profiles such as:

- 60/30/10
- 50/40/10
- 33/33/34
- 25/65/10
- 80/15/5

Each profile represents a possible Gold/Silver/Bronze bandwidth allocation. For each profile, the recommender rewrites the queue-weight features, predicts class-level delay, checks SLA thresholds, and ranks the profiles. The reported allocation results use the trained XGBoost delay model; running the same workflow with the neural models is left as future work. This directly addresses the original SD-WAN question: how much bandwidth should be allocated to each QoS class?

# Chapter 5

## Experiments and Results

This chapter presents the experiments completed so far. The results are organised by the three project layers: Zenodo throughput prediction, BNN-UPC per-class delay prediction, and Layer 3 QoS allocation recommendation.

### 5.1 Layer 1: Zenodo Throughput Prediction

#### 5.1.1 Dataset and Setup

The Zenodo 5G campus network dataset [5] was used to predict achieved throughput. The processed dataset contains 604 rows after uplink and downlink measurements are split into separate records. The target is `actual_throughput_mbps`.

The dataset was split into 80% training and 20% testing using a fixed random seed. Five-fold cross-validation was also used on the training split. Leakage columns such as `recommended_bandwidth_percent`, `jitter_ms`, and `packet_loss_percent` were dropped. Constant metadata columns such as `protocol`, `application_type`, `link_type`, and `sdr` were also removed because they do not vary across the dataset.



---

## 5.2 Layer 2: BNN-UPC Delay Prediction

### 5.1.2 Results

Table 5.1 shows the Zenodo throughput results after dropping constant metadata features.

Table 5.1: Zenodo Throughput Prediction Results (Target: `actual_throughput_mbps`)

| Model             | MAE   | RMSE  | $R^2$  | CV $R^2$ |
|-------------------|-------|-------|--------|----------|
| DummyRegressor    | 36.90 | 43.88 | -0.007 | -0.011   |
| Linear Regression | 10.55 | 14.15 | 0.895  | 0.851    |
| SVR (RBF)         | 5.27  | 8.16  | 0.965  | 0.925    |
| Random Forest     | 2.76  | 6.13  | 0.980  | 0.936    |
| XGBoost           | 3.20  | 6.32  | 0.979  | 0.934    |

The Zenodo results show that supervised ML can predict achieved throughput well from radio and traffic configuration features. Random Forest gives the best held-out  $R^2$ , while XGBoost is close. However, this dataset contains only one application type and no Gold/Silver/Bronze class variation. Therefore, it is useful as a link-capacity prediction experiment, but it cannot answer the per-class QoS allocation question on its own.

## 5.2 Layer 2: BNN-UPC Delay Prediction

### 5.2.1 Dataset and Setup

The BNN-UPC dataset was generated using BNNetSimulator. It contains 29,280 per-flow records. Each flow has a QoS class label: Gold, Silver, or Bronze. The target used for modelling is `log_avg_delay`, because raw delay is heavy-tailed.

The feature set includes offered bandwidth, ToS, QoS class, routing hops,

---

## 5.2 Layer 2: BNN-UPC Delay Prediction

topology size, traffic distribution, scheduling policy, and queue weights. Outcome columns such as raw delay, jitter, loss rate, delay percentiles, and actual bandwidth were removed from model inputs.

### 5.2.2 Global Model Results

Table 5.2 shows the global BNN-UPC results on `log_avg_delay`.

Table 5.2: BNN-UPC Global Results (Target: `log_avg_delay`)

| Model             | MAE   | RMSE  | $R^2$ | CV $R^2$ |
|-------------------|-------|-------|-------|----------|
| Linear Regression | 0.227 | 0.415 | 0.685 | 0.690    |
| SVR (RBF)         | 0.174 | 0.391 | 0.721 | 0.721    |
| Random Forest     | 0.181 | 0.364 | 0.758 | 0.764    |
| XGBoost           | 0.185 | 0.359 | 0.765 | 0.766    |
| PyTorch MLP       | 0.185 | 0.357 | 0.766 | 0.769    |

The PyTorch MLP is marginally ahead of XGBoost on the global score, but the difference is very small and within cross-validation noise: the five-fold cross-validated  $R^2$  is  $0.766 \pm 0.008$  for XGBoost and  $0.769 \pm 0.008$  for the MLP (mean  $\pm$  standard deviation across folds). The fold-level variability is much larger than the mean difference. This means the neural-network path is competitive, while XGBoost remains an important explainable tree-based baseline for this tabular dataset. The useful comparison is therefore not only which model has the highest global score, but also whether the MLP and XGBoost behave similarly for each QoS class. Section 5.3.4.1 tests this difference formally.

## 5.3 QoS-Aware Evaluation

A single global metric is not enough for BNN-UPC. Bronze delay is much more volatile than Gold and Silver delay. If all rows are pooled together, Bronze errors can dominate the score and hide how the model behaves for high-priority traffic.

Therefore, the XGBoost model was evaluated using out-of-fold predictions and sliced by QoS class, scheduling scenario, and scheduling policy. This provides the current explainable baseline for class-aware evaluation. The same class-aware evaluation was then applied to the PyTorch MLP so that the neural model is assessed per class, not only through a global score.

### 5.3.1 Per-Class Results

Table 5.3 shows per-class results. The metric values are on the delay scale in milliseconds, except  $R^2$ , which is computed on the log-delay target.

Table 5.3: BNN-UPC Per-Class Evaluation using XGBoost

| QoS Class | Rows   | MAE Delay | RMSLE | $R^2$ Log Delay |
|-----------|--------|-----------|-------|-----------------|
| Gold      | 4,592  | 3.05 ms   | 0.160 | 0.914           |
| Silver    | 8,991  | 3.48 ms   | 0.168 | 0.908           |
| Bronze    | 15,697 | 20.68 ms  | 0.451 | 0.704           |

This result is important. Gold and Silver are predicted accurately, but Bronze is much harder. This confirms that global model scores alone are not enough for QoS evaluation.

### 5.3.2 Policy and Scenario Results

Table 5.4 shows that the mixed-policy scenario is the hardest to predict.

---

### 5.3 QoS-Aware Evaluation

Table 5.4: BNN-UPC Scenario-Specific Evaluation using XGBoost

| Scenario       | Rows  | MAE Delay | $R^2$ Log Delay |
|----------------|-------|-----------|-----------------|
| A_wfq_fixed    | 6,560 | 6.16 ms   | 0.796           |
| B_wfq_profiles | 8,240 | 12.01 ms  | 0.758           |
| C_mixed_policy | 7,600 | 21.32 ms  | 0.743           |
| D_mixed_equal  | 6,880 | 9.95 ms   | 0.779           |

Table 5.5 shows results sliced by scheduling policy. Strict Priority (SP) achieves the best  $R^2$  because SP flows follow a deterministic priority order that the model can learn well. WFQ is harder because queue weights vary across profiles and interact with traffic load in more complex ways.

Table 5.5: BNN-UPC Scheduling Policy Evaluation using XGBoost (v1 Dataset). Policies: SP (Strict Priority), DRR (Deficit Round Robin), WFQ (Weighted Fair Queueing).

| Policy | Rows   | MAE (ms) | RMSLE | $R^2$ |
|--------|--------|----------|-------|-------|
| SP     | 8,120  | 8.27     | 0.282 | 0.806 |
| DRR    | 1,840  | 8.01     | 0.302 | 0.678 |
| WFQ    | 19,320 | 14.90    | 0.378 | 0.756 |

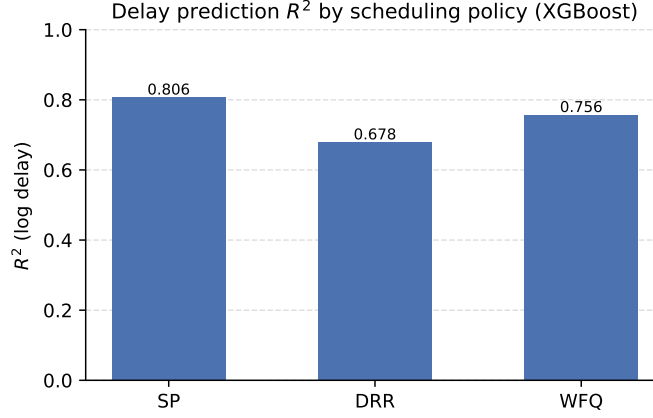


Figure 5.1: Delay prediction  $R^2$  by scheduling policy (XGBoost). Strict Priority is the easiest to predict because its priority order is deterministic; DRR is the hardest.

### 5.3.3 SLA Violation Precision

For SD-WAN use, exact delay prediction is not the only concern. The model should also identify when a flow is likely to violate an SLA. The continuous delay prediction was converted into a binary trigger using these thresholds: Gold  $> 30$  ms, Silver  $> 50$  ms, and Bronze  $> 60$  ms.

The Bronze threshold was tuned from an initial value of 100 ms to 60 ms following a sensitivity sweep. Table 5.6 summarises how precision, recall, and F1 change as the Bronze threshold is lowered. Lowering the threshold increases the number of flows treated as violations in the trigger evaluation, making the alert more sensitive and improving recall. The 60 ms value was chosen because it keeps Bronze strictly more lenient than Silver (50 ms) while significantly improving recall.

### 5.3 QoS-Aware Evaluation

Table 5.6: Bronze SLA Threshold Sensitivity (Gold=30 ms, Silver=50 ms fixed)

| Bronze Threshold | Precision | Recall | F1    |
|------------------|-----------|--------|-------|
| 100 ms           | 0.726     | 0.368  | 0.488 |
| 70 ms            | 0.614     | 0.476  | 0.536 |
| 60 ms            | 0.619     | 0.568  | 0.592 |
| 50 ms            | 0.582     | 0.735  | 0.650 |

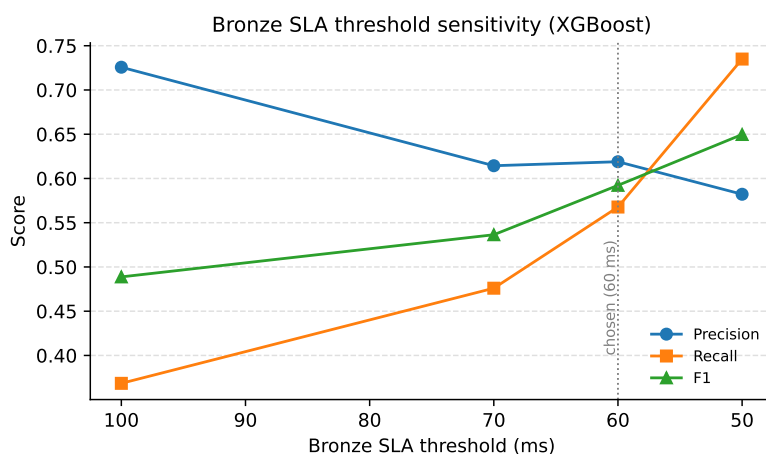


Figure 5.2: Bronze SLA threshold sensitivity for XGBoost. As the threshold is lowered from left to right, recall rises and precision falls. The chosen 60 ms value balances the two while staying more lenient than Silver (50 ms).

Table 5.7 shows the final SLA violation trigger results using the tuned thresholds.

Table 5.7: SLA Violation Trigger Results using XGBoost (Tuned Thresholds)

| QoS Class | Threshold | Precision | Recall | F1    |
|-----------|-----------|-----------|--------|-------|
| Gold      | 30 ms     | 0.994     | 0.899  | 0.944 |
| Silver    | 50 ms     | 0.823     | 0.639  | 0.720 |
| Bronze    | 60 ms     | 0.619     | 0.568  | 0.592 |

Gold precision remains very high (0.994), meaning the model is correct 99.4% of the time when it predicts a Gold SLA violation. This is critical for SD-WAN policy triggers, because false positives for Gold could cause unnecessary rerouting of high-priority traffic. Bronze recall improves from 0.368 (at 100 ms) to 0.568 (at 60 ms), a 55% improvement, while precision drops only modestly from 0.726 to 0.619. The threshold adjustment makes the alerting system more sensitive for Bronze, which is appropriate given that Bronze is best-effort traffic and missing a violation is more acceptable than for Gold.

### 5.3.4 Model Comparison: XGBoost vs MLP vs FT-Transformer

The results above use XGBoost as the strongest explainable tree-based baseline. To evaluate the neural-network direction of the project, the same QoS-aware analysis was repeated for two neural models of increasing sophistication:

- A **PyTorch feedforward MLP** (three hidden layers of 128, 64, and 32 units, ReLU activations, dropout 0.1, AdamW optimiser, early stopping).
- An **FT-Transformer** (Feature Tokenizer + Transformer), a modern attention-based architecture designed specifically for tabular data. Each feature (numeric or categorical) is converted into a learned token, a [CLS] token is prepended, three pre-norm Transformer blocks (eight attention heads, token dimension 64) apply self-attention across the feature tokens, and the final [CLS] representation is mapped to the predicted delay.

For all three models the outer out-of-fold evaluation protocol and metric functions are identical; the two neural models additionally use an internal validation split inside each training fold for early stopping. This makes the per-class num-

### 5.3 QoS-Aware Evaluation

---

bers directly comparable while avoiding validation leakage. Including the FT-Transformer directly addresses the question of whether a modern deep-learning architecture, rather than a plain MLP, can outperform the tree-based baseline on this problem.

Table 5.8 compares per-class delay prediction.

Table 5.8: Per-Class Delay Prediction: XGBoost vs MLP vs FT-Transformer (v1 Dataset, 5-fold out-of-fold)

| QoS Class | Model          | MAE (ms) | RMSLE | $R^2$ |
|-----------|----------------|----------|-------|-------|
| Gold      | XGBoost        | 3.05     | 0.160 | 0.914 |
| Gold      | MLP            | 3.08     | 0.161 | 0.912 |
| Gold      | FT-Transformer | 3.20     | 0.172 | 0.900 |
| Silver    | XGBoost        | 3.48     | 0.168 | 0.908 |
| Silver    | MLP            | 3.42     | 0.169 | 0.908 |
| Silver    | FT-Transformer | 3.53     | 0.178 | 0.898 |
| Bronze    | XGBoost        | 20.68    | 0.451 | 0.704 |
| Bronze    | MLP            | 20.60    | 0.450 | 0.705 |
| Bronze    | FT-Transformer | 20.52    | 0.452 | 0.703 |

On delay prediction all three models are practically very similar: per-class  $R^2$  agrees to within about 0.01, and on the hardest class (Bronze) all three land at  $R^2 \approx 0.70$ . Neither neural model beats XGBoost; the FT-Transformer is in fact marginally behind on Gold and Silver (with reasonable but not exhaustive hyper-parameter tuning). This is consistent with the well-documented finding that gradient-boosted trees remain highly competitive with, and often superior to, deep-learning models on small, static, tabular feature sets such as this one.



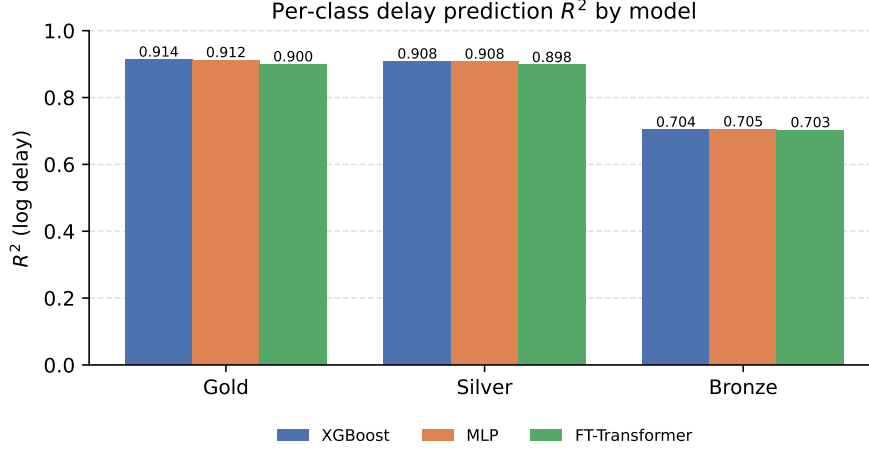


Figure 5.3: Per-class delay prediction  $R^2$  for the three models. All three converge to almost identical performance per class, and all reach the same ceiling on the hard Bronze class ( $R^2 \approx 0.70$ ), indicating that the limiting factor is the static feature set rather than model capacity.

The most important observation is that all three model families converge to the same per-class ceiling. This strongly suggests that the limiting factor is the *feature set*, not the choice of model: the static configuration features explain Gold and Silver delay very well but cannot fully capture Bronze’s burst-driven variance, regardless of model capacity.

Table 5.9 compares SLA violation detection at the tuned thresholds.

---

### 5.3 QoS-Aware Evaluation

Table 5.9: SLA Violation Detection: XGBoost vs MLP vs FT-Transformer (Tuned Thresholds)

| QoS Class | Threshold | Model          | Precision | Recall | F1    |
|-----------|-----------|----------------|-----------|--------|-------|
| Gold      | 30 ms     | XGBoost        | 0.994     | 0.899  | 0.944 |
| Gold      | 30 ms     | MLP            | 0.979     | 0.908  | 0.942 |
| Gold      | 30 ms     | FT-Transformer | 0.980     | 0.898  | 0.937 |
| Silver    | 50 ms     | XGBoost        | 0.823     | 0.639  | 0.720 |
| Silver    | 50 ms     | MLP            | 0.791     | 0.715  | 0.751 |
| Silver    | 50 ms     | FT-Transformer | 0.839     | 0.583  | 0.688 |
| Bronze    | 60 ms     | XGBoost        | 0.619     | 0.568  | 0.592 |
| Bronze    | 60 ms     | MLP            | 0.591     | 0.608  | 0.599 |
| Bronze    | 60 ms     | FT-Transformer | 0.597     | 0.580  | 0.589 |

On SLA detection the three models again behave similarly, with only small differences in the precision–recall balance. The MLP leans slightly towards higher recall on the harder Silver and Bronze classes, while XGBoost and the FT-Transformer are marginally more precision-oriented. Across all three models Gold precision stays very high (0.98–0.99), which is the most important property for SD-WAN policy triggers because false positives on high-priority traffic would cause unnecessary rerouting.

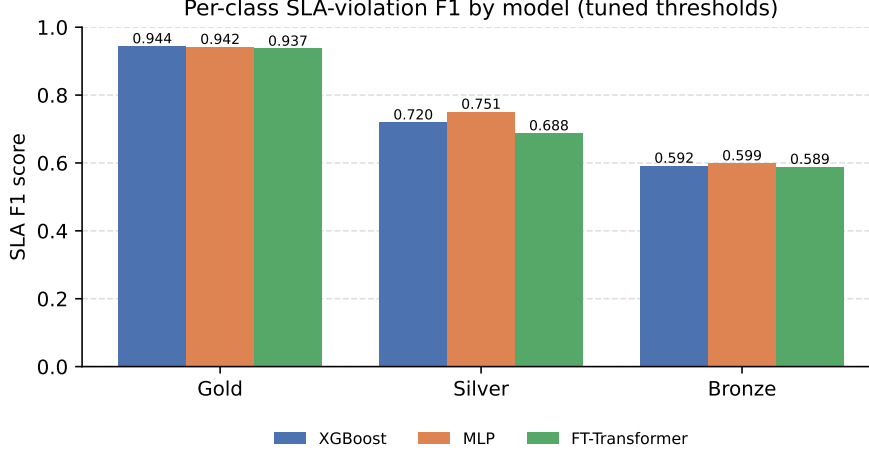


Figure 5.4: Per-class SLA-violation F1 score for the three models at the tuned thresholds (Gold 30 ms, Silver 50 ms, Bronze 60 ms). The models behave similarly; differences reflect small shifts in the precision–recall balance rather than a clear winner.

#### 5.3.4.1 Statistical Significance of the XGBoost–MLP Difference

To test whether the small global gap between XGBoost and the MLP is real, both models were trained on the *same* five cross-validation folds and their per-fold  $R^2$  values were compared with paired tests. This fold-level paired comparison is the standard way to compare two learners under cross-validation; a per-row test was deliberately avoided because, with roughly 29,000 rows, even a negligible difference would register as significant.

The MLP was higher by a mean of only 0.0006  $R^2$  (XGBoost 0.765, MLP 0.766 averaged over the five folds). A paired  $t$ -test returned  $p = 0.620$ , while the rank-based Wilcoxon signed-rank test returned  $p = 0.8125$ . Both tests are non-significant at the 0.05 level, and the difference (0.0006) is far smaller than the fold-to-fold standard deviation ( $\approx 0.016$ ). The practical conclusion is that the two models are equivalent for this task: any difference is an order of magnitude

---

## 5.4 Multi-Metric QoS Evaluation (v1 Dataset)

smaller than normal resampling variation, and with only five folds the tests have limited power. This supports, with evidence rather than by inspection, the claim that the neural model does not meaningfully outperform the tree-based baseline.

Overall, XGBoost is retained as the recommended model: it matches or slightly exceeds both neural models on accuracy while offering interpretability (feature importance) and substantially lower training cost (seconds versus minutes per fold for the FT-Transformer). The progression from a tree-based baseline through a feedforward MLP to a modern tabular Transformer confirms that the strong per-class results are a property of the data and features, not an artefact of a single model family, and that added model complexity does not improve QoS delay prediction on this dataset.

## 5.4 Multi-Metric QoS Evaluation (v1 Dataset)

Beyond average delay, the BNN-UPC v1 dataset was used to evaluate delay\_p90 and jitter per QoS class. The same out-of-fold XGBoost pipeline was used for both targets, with a log transform applied before training.

### 5.4.1 delay\_p90 Results

Table 5.10 shows the per-class results for 90th-percentile delay prediction.

Table 5.10: Per-Class delay\_p90 Prediction Results using XGBoost (v1 Dataset)

| QoS Class | Rows   | MAE (ms) | RMSLE | $R^2$ |
|-----------|--------|----------|-------|-------|
| Gold      | 4,592  | 6.64 ms  | 0.205 | 0.635 |
| Silver    | 8,991  | 7.29 ms  | 0.215 | 0.657 |
| Bronze    | 15,697 | 38.22 ms | 0.499 | 0.258 |

Gold and Silver show moderate predictability ( $R^2 \approx 0.64$ – $0.66$ ), which is lower

than for average delay. This is expected because the 90th percentile captures rarer, more variable congestion events. Bronze tail delay is much harder to predict ( $R^2 = 0.258$ ), following the same pattern as average delay but amplified.

### 5.4.2 Jitter Results

Table 5.11 shows the per-class results for jitter prediction.

Table 5.11: Per-Class Jitter Prediction Results using XGBoost (v1 Dataset)

| QoS Class | Rows   | MAE (ms) | RMSLE | $R^2$ |
|-----------|--------|----------|-------|-------|
| Gold      | 4,592  | 0.14 ms  | 0.176 | 0.119 |
| Silver    | 8,991  | 0.16 ms  | 0.190 | 0.084 |
| Bronze    | 15,697 | 3.11 ms  | 0.678 | 0.014 |

Jitter  $R^2$  is very low across all classes (0.01–0.12). This is a meaningful finding rather than a modelling failure. The BNN-UPC features describe static network configuration: queue weights, scheduling policy, and offered bandwidth. These features determine mean delay well but do not capture the moment-to-moment traffic burstiness that drives jitter. Jitter prediction would require dynamic traffic features such as instantaneous queue occupancy or inter-arrival time statistics, which are not present in this dataset. This is discussed further as a limitation and future work direction.

## 5.5 Bronze Packet Loss Analysis

In the v1 dataset, packet loss is realistic: Gold and Silver flows experience near-zero loss (4 and 93 loss events respectively out of 13,583 combined rows), while Bronze flows experience non-zero loss in 1,684 out of 15,697 rows (10.7%). This

matches the expected behaviour of a network where high-priority traffic (Gold, Silver) is protected and only best-effort Bronze traffic suffers under load.

Following supervisor guidance, packet loss evaluation is restricted to Bronze traffic only. Gold and Silver loss events are too rare and too small to train a reliable model on, and artificially forcing loss onto high-priority classes would not reflect a realistic network scenario.

### 5.5.1 Regression Attempt

Regression on `packet_loss_rate` for Bronze flows produced  $R^2 = -0.130$ . This is a meaningful finding: packet loss events in this dataset are driven by rare, bursty buffer overflow events. The static network configuration features (queue weights, scheduling policy, offered bandwidth) determine mean delay well but cannot capture when a particular burst will exceed the buffer. Predicting the exact loss rate requires dynamic features such as instantaneous buffer occupancy or inter-arrival time statistics, which are not present in the dataset.

### 5.5.2 Binary Loss Classification for Bronze

A more appropriate framing is binary classification: predicting whether a Bronze flow will experience any packet loss at all ( $\text{loss} > 0$ ). This converts the problem from estimating an unpredictable rate to a detection task. The class balance is 89.3% no-loss versus 10.7% loss, so class weighting (`scale_pos_weight = 8.3`) is applied.

Table 5.12 shows 5-fold stratified cross-validation results for XGBoost and Random Forest.

---

## 5.6 Layer 3: Allocation Recommender

Table 5.12: Bronze Binary Loss Classification Results (v1 Dataset, 5-fold CV)

| Model                       | Precision | Recall | F1    | AUC-ROC |
|-----------------------------|-----------|--------|-------|---------|
| XGBoost (default threshold) | 0.777     | 0.328  | 0.461 | 0.914   |
| XGBoost (class-weighted)    | 0.342     | 0.865  | 0.490 | 0.916   |
| Random Forest (balanced)    | 0.663     | 0.389  | 0.490 | 0.895   |

The AUC-ROC of 0.916 for class-weighted XGBoost confirms the model has strong discriminative ability: it can rank Bronze flows by loss probability effectively. The low recall at the default 0.5 decision threshold (0.328) is a threshold choice, not a model failure. Using class weighting raises recall to 0.865 at the cost of lower precision (0.342), which may be appropriate for an alerting system where missing a loss event is more harmful than a false alarm.

This shows that while exact loss-rate regression is not feasible from static features, the model can reliably identify which Bronze flows are at elevated risk of experiencing packet loss.

## 5.6 Layer 3: Allocation Recommender

The final experiment uses the trained BNN-UPC delay model as a surrogate model for allocation decisions. The recommender is designed so that it can use either an explainable model such as XGBoost or a neural model such as the PyTorch MLP. The current reported allocation results use the trained delay model already evaluated in this chapter. Several WFQ profiles were tested: 60/30/10, 50/40/10, 33/33/34, 25/65/10, and 80/15/5. Each profile was applied to the traffic pattern, and the model predicted the expected delay for Gold, Silver, and Bronze.

The default SLA thresholds for this allocation experiment were Gold  $< 20$

---

## 5.7 Summary of Findings

ms, Silver < 50 ms, and Bronze < 100 ms. This recommender run keeps the original allocation thresholds; rerunning the recommender with the tuned Bronze threshold of 60 ms is left as a follow-up comparison.

Table 5.13: Layer 3 WFQ Allocation Recommendation Results

| Rank | Profile  | Feasible | Gold Mean | Silver Mean | Bronze Mean |
|------|----------|----------|-----------|-------------|-------------|
| 1    | 60/30/10 | No       | 24.47 ms  | 24.22 ms    | 30.81 ms    |
| 2    | 80/15/5  | No       | 24.99 ms  | 30.62 ms    | 31.27 ms    |
| 3    | 25/65/10 | No       | 25.02 ms  | 24.28 ms    | 30.81 ms    |
| 4    | 33/33/34 | No       | 25.12 ms  | 24.55 ms    | 33.92 ms    |
| 5    | 50/40/10 | No       | 25.27 ms  | 24.80 ms    | 30.81 ms    |

No tested profile fully satisfies the strict Gold threshold of 20 ms. The recommender therefore selects 60/30/10 as the least-violating profile. This is still useful because the system can show both a recommendation and an infeasibility warning. In practice, this could tell an SD-WAN controller that changing WFQ weights alone may not be enough and that rerouting or extra capacity may be needed. A future comparison should run the same profile search using the PyTorch MLP and check whether the neural model recommends the same allocation.

## 5.7 Summary of Findings

The completed experiments show seven main findings:

- Zenodo is useful for link capacity prediction, but not for per-class QoS allocation.
- BNN-UPC provides a useful per-class QoS scheduling dataset. Global metrics hide class-level differences; Bronze is much harder to predict than Gold



and Silver.

- QoS-aware evaluation with per-class MAE, RMSLE, and SLA violation precision is essential for reporting honest model behaviour for SD-WAN.
- `delay_p90` is moderately predictable ( $R^2 \approx 0.64$ – $0.66$  for Gold/Silver), but jitter is not predictable from static configuration features ( $R^2 < 0.12$ ). This is a feature gap, not a modelling failure.
- Bronze SLA recall improves significantly when the violation threshold is tuned from 100 ms to 60 ms (Recall:  $0.368 \rightarrow 0.568$ , F1:  $0.489 \rightarrow 0.592$ ). The 60 ms threshold was selected to keep Bronze strictly more lenient than Silver (50 ms) while providing a meaningful recall improvement.
- Packet loss is only meaningful for Bronze traffic in a realistic network. Regression on `packet_loss_rate` is not feasible from static features ( $R^2 = -0.130$ ), but binary classification (loss vs. no loss) achieves AUC-ROC = 0.916 for Bronze flows, confirming the model can identify loss-prone flows reliably.
- Three model families were compared per class under an identical evaluation protocol: XGBoost, a feedforward MLP, and a modern attention-based FT-Transformer. All three converge to the same per-class ceiling (Bronze  $R^2 \approx 0.70$  for every model), and neither neural model beats XGBoost. This shows the limiting factor is the static feature set, not model capacity, and XGBoost is retained as the recommended model for its equal accuracy, interpretability, and far lower training cost.

# Chapter 6

## Limitations

This chapter states the main limitations of the work explicitly, so that the results are not over-interpreted.

### 6.1 Data Availability and Realism

No real-world enterprise SD-WAN dataset with labelled QoS metrics was available for this project, so the work relied on public and simulator-generated data. The Zenodo 5G dataset provides real measurements but contains no Gold/Silver/Bronze QoS-class structure. The BNN-UPC dataset does provide per-class behaviour, but it is produced by a network simulator (BNNetSimulator) rather than measured on a live network. The official BNN-UPC download links were unavailable, so the dataset was generated locally using the official simulator and the same output format. CICIDS2017 was discarded because it is an intrusion-detection dataset rather than a QoS dataset, and MAWI was not used because it lacks direct QoS-class labels. The findings should therefore be read as a reproducible proof of concept, not as a claim of production-grade generalisation.

## 6.2 Static Features and the Bronze Ceiling

All models were trained on static configuration features (offered load, routing hops, scheduling policy, queue weights, and ToS class). These features explain Gold and Silver delay well, but they cannot capture the short-term traffic burstiness that drives Bronze delay variance, tail delay, jitter, and packet loss. This limit was confirmed empirically: three different model families, and additional engineered interaction features, all converged to the same per-class ceiling (Bronze  $R^2 \approx 0.70$ ). The limitation is therefore one of feature observability rather than model capacity.

## 6.3 Offline Evaluation

The evaluation is entirely offline. The metrics used (per-class error, RMSLE, SLA-violation precision and recall, and the allocation recommender) are closer to operational SD-WAN use than a single global score, but they do not capture live behaviour such as failover, multi-path routing, or the feedback effects of acting on a prediction. Validation inside a live SD-WAN controller remains future work.

## 6.4 Computational Constraints

Model training and tuning were carried out on a single laptop without dedicated GPU acceleration. This limited the extent of neural-network hyper-parameter search and the size of the simulated dataset. Because the tree-based and neural models reached the same per-class ceiling, this constraint is unlikely to have changed the main conclusions, but a larger compute budget would allow a wider search and more simulated data.

# Chapter 7

## Conclusion

This thesis investigated the use of machine learning models to predict Quality of Service (QoS) outcomes in SD-WAN-like network environments. The final project has three layers: link capacity prediction, per-class delay prediction, and allocation recommendation.

The literature review identified a broad range of AI and ML techniques applied to QoS prediction and SD-WAN traffic engineering, ranging from classical supervised learning to graph neural networks and reinforcement learning. The review confirmed that while deep learning and transformer-based models represent the current state of the art, interpretable baseline models remain essential as comparison benchmarks and are computationally feasible for near-real-time deployment.

The first dataset is the Zenodo 5G campus network dataset. It is useful for predicting link capacity from radio and traffic features. On this dataset, tree-based and kernel models perform strongly, with Random Forest and XGBoost reaching a held-out test  $R^2$  of about 0.98 (cross-validated  $R^2 \approx 0.94$ ). This shows that the measured link conditions contain a strong signal for throughput prediction.

---

The second dataset is the BNN-UPC dataset generated using BNNetSimulator. This dataset is more important for the original research question because it includes Gold, Silver, and Bronze traffic classes under different scheduling policies and queue weights. On log-transformed average delay all evaluated models reach a similar global score (cross-validated  $R^2 \approx 0.77$ ). A per-class comparison of three model families, XGBoost, a feedforward MLP, and an FT-Transformer, showed that they converge to almost identical per-class performance, and neither neural model beat XGBoost. XGBoost is therefore retained as the recommended model because it matches the neural models while remaining interpretable and far cheaper to train.

The QoS-aware evaluation is a key part of the work. It shows that a single global score is not enough. Gold and Silver delay are predicted with low error, while Bronze is more difficult because its delay is more volatile and heavy-tailed. SLA violation precision is especially strong for Gold traffic, which is useful because Gold flows are normally the most important class for SD-WAN policy decisions.

The Layer 3 allocation recommender uses the trained BNN-UPC delay model to compare candidate WFQ weight profiles. In the tested traffic pattern, none of the profiles fully satisfy the strict Gold SLA threshold of 20 ms. The recommender therefore selects the least-violating profile and reports that weight adjustment alone may not be enough. This connects the predictive model back to the original research question: how much bandwidth should be allocated to each QoS class? The current reported allocation results should be extended by running the same recommender with the PyTorch MLP and comparing the per-class allocation decisions against XGBoost.

### 7.1 Real-World Impact or Benefit

The core value of this project lies in supporting proactive Quality of Service (QoS) management in SD-WAN environments through AI-driven predictions. In current networks, QoS violations are often addressed reactively, after user experience is already affected. This work is a step in that direction: it predicts per-class QoS behaviour from network configuration features offline, as a proof of concept that could later be driven by live telemetry.

**Potential benefits, if integrated into a live system:**

- **Proactive Network Management:** Helps operators act before congestion or SLA violations occur.
- **Cost Optimization:** Enables more informed bandwidth allocation and may reduce unnecessary overprovisioning.
- **Improved QoE (Quality of Experience):** Real-time predictions allow better support for critical applications like video calls, VoIP, and live collaboration tools.
- **Scalability:** The approach is suitable for large enterprise SD-WANs that span multiple locations and require dynamic QoS adjustments.

### 7.2 Future Work

Based on the current findings, future research should explore the following directions:

- **Bronze Loss Classifier Deployment:** The current Bronze binary loss classifier achieves  $\text{AUC-ROC} = 0.916$ . The next step is to select the optimal decision threshold (e.g. using precision-recall curve analysis) and integrate

the classifier alongside the delay model so that the SD-WAN controller receives both a delay prediction and a loss-risk score for each Bronze flow.

- **Neural Allocation Recommender:** Extend the Layer 3 recommender to use the PyTorch MLP, then compare per-class allocation results against XGBoost using Gold/Silver/Bronze SLA thresholds.
- **Better Bronze Delay Prediction:** Bronze delay is harder to predict than Gold and Silver ( $R^2 = 0.704$  vs. 0.914 and 0.908). A dedicated model with class-weighted training or a separate Bronze-only model may reduce the Bronze MAE further. The tuned SLA threshold (60 ms) already improves recall, but better per-class models remain worth exploring.
- **Jitter Prediction with Dynamic Features:** Jitter is not predictable from static configuration features. Future work should explore adding dynamic traffic features such as instantaneous queue depth or inter-arrival time statistics.
- **More Allocation Profiles:** Test a wider search space of WFQ weights and include rerouting or extra-capacity decisions when no feasible weight profile exists.
- **Real-Time Integration:** Integrate the predictive model into an SD-WAN controller or policy engine.
- **Cross-Domain Generalisation:** Test the approach on more topologies, real traffic traces, or additional public datasets such as MAWI.

By addressing these improvements, the project could evolve into a plug-in or microservice within SD-WAN platforms that provides predictive QoS as a built-in capability.

### 7.3 Final Remarks

AI and ML hold significant promise for improving QoS in SD-WAN by enabling proactive, predictive management rather than reactive troubleshooting. While the current body of research is still emerging and somewhat fragmented, there is a strong foundation to build upon. With more focused work and collaboration between academia and industry, AI-driven QoS prediction can become a core feature of future network systems.



# References

- [1] Yuyu Chen, Anyang Gu, Lin Cui, and Longxin Lin. MTEAL: Network routing optimization of SD-WAN traffic engineering integrating multi-dimensional QoS metrics. *Journal of Network and Computer Applications*, 242:104272, 2025. doi: 10.1016/j.jnca.2025.104272.
- [2] Lam Dinh, Pham Tran Anh Quang, and Jérémie Leguay. Towards safe load balancing based on control barrier functions and deep reinforcement learning. *arXiv preprint arXiv:2401.05525*, 2024.
- [3] Pham Tran Anh Quang, Xu Gong, Chuang Liu, Kun Li, Jérémie Leguay, Xinhui Zhang, Yong Zhang, Jiayan Li, and Kai Ye. Routing and qos policy optimization in sd-wan. In *Proceedings of the IEEE International Conference on Cloud Networking (CloudNet)*, pages 1–7. IEEE, 2022.
- [4] Pham Tran Anh Quang, Jérémie Leguay, Gong Xu, and Huiying Xu. Global qos policy optimization in sd-wan. *arXiv preprint arXiv:2304.05473*, 2023.
- [5] Simon Raffeck, Sebastian G. Grøsvik, Stanislav Lange, Tobias Hossfeld, Thomas Zinner, and Stefan Geissler. 5g campus network dataset: Throughput and one-way delay measurements, 2024. URL <https://zenodo.org/records/13754300>.
- [6] Sabidur Rahman, Tanjila Ahmed, Minh Huynh, Massimo Tornatore, and

## REFERENCES

---

- Biswanath Mukherjee. Auto-scaling vnfs using machine learning to improve qos and reduce cost. *IEEE Transactions on Network and Service Management*, 16(4):1458–1472, 2019. doi: 10.1109/TNSM.2019.2948372.
- [7] Xun Zheng, Jian Li, You Ma, Tianlong Zhang, and Xuan Du. Application-aware qos routing in sdns using machine learning techniques. *Peer-to-Peer Networking and Applications*, 15(3):1454–1466, 2022.